

ODOO VIEWS, SECURITY, AND QWEB REPORTS

Views, Security & QWeb

UI views, access rights, and report basics

Merit Advisory

Odoo Full-Stack Developer Program

Beginner-friendly lecture materials

July 2026

July 2026



Agenda

- 1 Section 1: List and Search Views — 10 topics
- 2 Section 2: Form and Analytical Views — 8 topics
- 3 Section 3: View Inheritance Positions and Groups — 8 topics
- 4 Section 4: Access Rights, Record Rules, and Data — 11 topics
- 5 Section 5: Reports, Templates, and Automation — 9 topics

1 Section 1

List and Search Views

1.1 List View Basics

- A list view shows many records in rows.
- Choose columns that help users decide what to open.
- Keep technical fields hidden unless users need them.

Simple list view

```
<tree string="Courses">  
  <field name="name"/>  
  <field name="teacher_id"/>  
  <field name="start_date"/>  
</tree>
```

List view action

```
<field name="view_mode">tree,form</field>  
<field name="res_model">training.course</field>
```

1.2 Editable List

- Editable lists let users change rows without opening a form.
- Use them for simple line data, not complex workflows.
- Always keep required fields visible or prefilled.

Edit at bottom

```
<tree editable="bottom">  
  <field name="product_id"/>  
  <field name="quantity"/>  
</tree>
```

Edit at top

```
<tree editable="top">  
  <field name="name"/>  
  <field name="score"/>  
</tree>
```

1.3 List Decorations

- Decorations color rows based on field values.
- They guide attention but do not replace real validation.
- Include invisible helper fields when the expression needs them.

Danger row

```
<tree decoration-danger="state == 'late'">  
  <field name="name"/>  
  <field name="state"/>  
</tree>
```

Hidden helper field

```
<field name="deadline_passed" invisible="1"/>  
<tree decoration-warning="deadline_passed">  
  <field name="name"/>  
</tree>
```

1.4 List Button

- A list button starts an action from a row.
- Use object buttons for Python model methods.
- Keep the label short because row space is limited.

Object button

```
<button name="action_confirm"  
        type="object"  
        string="Confirm"  
        class="btn-primary"/>
```

Button with state rule

```
<button name="action_reset"  
        type="object"  
        string="Reset"  
        invisible="state == 'draft'"/>
```

1.5 Sort and Limit

- default_order controls the first sort users see.
- limit controls how many rows load at first.
- Sort on fields that make business sense.

Newest first

```
<tree default_order="create_date desc" limit="80">  
  <field name="name"/>  
</tree>
```

Priority queue

```
<tree default_order="priority desc, date_deadline">  
  <field name="priority"/>  
  <field name="date_deadline"/>  
</tree>
```

1.6 Group By in Search

- Group by organizes records into collapsible buckets.
- Put common groupings in the search view.
- Use fields that users already understand.

Group by teacher

```
<filter string="Teacher"  
        name="group_teacher"  
        context="{ 'group_by': 'teacher_id' }"/>
```

Group by month

```
<filter string="Start Month"  
        name="group_start_month"  
        context="{ 'group_by': 'start_date:month' }"/>
```

1.7 Search View Basics

- A search view defines quick filters and search fields.
- It supports both typed search and saved filter buttons.
- Name filters clearly for non-technical users.

Search by fields

```
<search string="Courses">  
  <field name="name"/>  
  <field name="teacher_id"/>  
</search>
```

Simple filter

```
<filter string="Published"  
  name="published"  
  domain="(['is_published', '=', True)]"/>
```

1.8 Search Panel

- The search panel gives a left-side filter menu.
- It works well for categories, companies, and owners.
- Avoid panels with too many rarely used options.

Category panel

```
<searchpanel>  
  <field name="category_id"/>  
</searchpanel>
```

Multi-select panel

```
<searchpanel>  
  <field name="tag_ids" select="multi"/>  
</searchpanel>
```

1.9 Default Filter

- A default filter opens an action with a filter already active. **Action context**
- Use the action context to enable it.
- The key must match the filter name.

```
<field name="context">
  {'search_default_published': 1}
</field>
```

Matching filter

```
<filter string="Published"
  name="published"
  domain="(['is_published', '=', True)]"/>
```

1.10 Action View Modes

- view_mode controls which views users can switch between.
- Put the most common view first.
- Do not expose a view type that has no useful design.

List first

```
<field name="view_mode">tree,form,kanban</field>
```

Report first

```
<field name="view_mode">pivot,graph,tree</field>
```

2 Section 2

Form and Analytical Views

2.1 Form Sheet and Group

- A sheet gives the form a document-like area.
- Groups align labels and fields in readable columns.
- Place important fields near the top.

Basic form

```
<form string="Course">  
  <sheet>  
    <group>  
      <field name="name"/>  
      <field name="teacher_id"/>  
    </group>  
# ...
```

Two columns

```
<group>  
  <group><field name="start_date"/></group>  
  <group><field name="end_date"/></group>  
</group>
```

2.2 Notebook Pages

- A notebook splits a form into tabs.
- Use tabs for secondary details, not the main identity.
- Keep tab names short and task based.

Notebook with lines

```
<notebook>
  <page string="Students">
    <field name="student_ids"/>
  </page>
</notebook>
```

Notes tab

```
<page string="Notes">
  <field name="description"/>
</page>
```

2.3 Statusbar

- A statusbar shows the record stage.
- It usually uses a selection field named state.
- Pair it with buttons that move the workflow forward.

State field

```
<field name="state"  
      widget="statusbar"  
      statusbar_visible="draft,confirmed,done"/>
```

Workflow button

```
<button name="action_confirm"  
        type="object"  
        string="Confirm"  
        invisible="state != 'draft'"/>
```

2.4 Kanban Basics

- Kanban views show records as cards.
- They are useful for pipelines, tasks, and visual queues.
- Keep each card focused on one quick decision.

Small card

```
<kanban>
  <templates>
    <t t-name="kanban-box">
      <div><field name="name"/></div>
    </t>
  </templates>
# ...
```

Group by stage

```
<kanban default_group_by="stage_id">
  <field name="stage_id"/>
</kanban>
```

2.5 Graph View

- A graph view turns records into charts.
- Use measures for numbers and grouping fields for axes.
- Start simple before adding many dimensions.

Bar chart

```
<graph string="Enrollments" type="bar">  
  <field name="teacher_id"/>  
  <field name="student_count" type="measure"/>  
</graph>
```

Line chart

```
<graph string="Revenue" type="line">  
  <field name="date" interval="month"/>  
  <field name="amount_total" type="measure"/>  
</graph>
```

2.6 Pivot View

- A pivot view helps users summarize records.
- Rows and columns are dimensions.
- Measures are numeric values to total or count.

Pivot by teacher

```
<pivot string="Course Analysis">  
  <field name="teacher_id" type="row"/>  
  <field name="student_count" type="measure"/>  
</pivot>
```

Pivot by month

```
<pivot string="Monthly Sales">  
  <field name="date_order" interval="month" type="col"/>  
  <field name="amount_total" type="measure"/>  
</pivot>
```

2.7 Invisible, Readonly, and Required

- View attributes guide user input in the client.
- They improve usability but do not replace server rules.
- Keep expressions easy to read.

Conditional readonly

```
<field name="teacher_id"  
      readonly="state != 'draft'"/>
```

Conditional required

```
<field name="cancel_reason"  
      invisible="state != 'cancelled'"  
      required="state == 'cancelled'"/>
```

2.8 View Inheritance XPath

- XPath lets one module modify another module's view.
- Use stable anchors such as field names or page names.
- Keep inherited changes small and easy to review.

Find a field

```
<xpath expr="//field[@name='name']" position="after">  
  <field name="code"/>  
</xpath>
```

Find a page

```
<xpath expr="//page[@name='settings']" position="inside">  
  <group string="Training"/>  
</xpath>
```

3 Section 3

View Inheritance Positions and Groups

3.1 Position After

- position='after' inserts new XML after the matched node.
- It is common for adding a field beside an existing field.
- Use it when order matters for the user.

After a field

```
<xpath expr="//field[@name='name']" position="after">  
  <field name="short_code"/>  
</xpath>
```

After a button

```
<xpath expr="//button[@name='action_confirm']" position="a  
  <button name="action_print" type="object" string="Prin  
</xpath>
```

3.2 Position Before

- position='before' inserts new XML before the matched node.
- It is useful when the new field should be seen first.
- Avoid moving too much layout with one XPath.

Before date

```
<xpath expr="//field[@name='start_date']" position="before">  
  <field name="priority"/>  
</xpath>
```

Before page

```
<xpath expr="//page[@name='notes']" position="before">  
  <page name="students" string="Students"/>  
</xpath>
```

3.3 Position Inside

- position='inside' adds content inside a matched container. **Inside group**
- It works with group, page, sheet, and header nodes.
- Make sure the target is a container, not a simple field.

```
<xpath expr="//group[@name='main']" position="inside">  
  <field name="level"/>  
</xpath>
```

Inside header

```
<xpath expr="//header" position="inside">  
  <button name="action_archive" type="object" string="Ar  
</xpath>
```

3.4 Position Replace

- position='replace' removes the matched node and inserts new XML.
- Use it carefully because it can break later inherited views.
- Prefer smaller changes when possible.

Replace a field

```
<xpath expr="//field[@name='description']" position="replace">  
    <field name="description" placeholder="Write notes here" />  
</xpath>
```

Remove by empty replace

```
<xpath expr="//field[@name='legacy_code']" position="replace">
```

3.5 Security Groups

- A group represents a security role.
- Users receive permissions through their groups.
- Use groups for roles, not for individual people.

Teacher group

```
<record id="group_training_teacher" model="res.groups">  
  <field name="name">Teacher</field>  
</record>
```

Student group

```
<record id="group_training_student" model="res.groups">  
  <field name="name">Student</field>  
</record>
```

3.6 Group Category

- A category groups related security roles in the user form. **Category record**
- It makes permissions easier to understand.
- Use a module-specific category for custom apps.

```
<record id="module_category_training" model="ir.module.category">  
  <field name="name">Training</field>  
  <field name="sequence">30</field>  
</record>
```

Group in category

```
<field name="category_id"  
  ref="module_category_training"/>
```

3.7 Implied Groups

- implied_ids automatically adds another group.
- Use it for role inheritance such as manager includes user.
- Keep the chain simple so access is predictable.

Manager implies teacher

```
<field name="implied_ids"  
      eval="[(4, ref('group_training_teacher')))]"/>
```

Base user implied

```
<field name="implied_ids"  
      eval="[(4, ref('base.group_user')))]"/>
```

3.8 Groups on Menu

- The groups attribute hides a menu from other users.
- It is a usability rule, not a full security rule.
- Pair menu groups with access rights and record rules.

Teacher menu

```
<menuitem id="training_menu_teacher"  
          name="Teacher Tools"  
          groups="training.group_training_teacher"/>
```

Manager menu

```
<menuitem id="training_menu_reports"  
          name="Reports"  
          groups="training.group_training_manager"/>
```

4 Section 4

Access Rights, Record Rules, and Data

4.1 ir.model.access.csv

- CSV access files grant model-level permissions.
- They control read, write, create, and delete.
- Every business model usually needs at least read access.

CSV header

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_
```

Teacher access row

```
access_course_teacher,course teacher,model_training_course
```

4.2 Access Rights XML

- Access rights can also be written as XML records.
- XML is useful when values need references or conditions.
- CSV is still the common choice for simple access matrices.

XML access record

```
<record id="access_course_teacher_xml" model="ir.model.access_record">  
  <field name="name">course teacher</field>  
  <field name="model_id" ref="model_training_course"/>  
  <field name="group_id" ref="group_training_teacher"/>  
  <field name="perm_read" eval="1"/>  
</record>
```

No delete access

```
<field name="perm_unlink" eval="0"/>  
<field name="perm_write" eval="1"/>
```

4.3 Record Rules

- Record rules filter which records a user can access.
- They apply after model access rights.
- Domains should be simple enough to audit.

Own records only

```
<field name="domain_force">  
  [['user_id', '=', user.id]]  
</field>
```

Company records only

```
<field name="domain_force">  
  [['company_id', 'in', company_ids]]  
</field>
```

4.4 Student Record Rule Example

- A student should usually see only their own enrollments. **Student enrollment rule**
- The rule links records to the current user's partner.
- Test with a real student account, not the administrator.

```
<record id="rule_student_own_enrollments" model="ir.rule">  
  <field name="name">Students see own enrollments</field>  
  <field name="model_id" ref="model_training_enrollment">  
    <field name="domain_force">[['student_id.partner_id',  
      <field name="groups" eval="[(4, ref('group_training_st  
</record>
```

Model access row

```
access_enrollment_student,enrollment student,model_training
```

4.5 Teacher Record Rule Example

- A teacher may see courses they teach.
- The rule should match the business relationship.
- Give broader access only to manager groups.

Teacher course rule

```
<record id="rule_teacher_own_courses" model="ir.rule">  
  <field name="name">Teachers see own courses</field>  
  <field name="model_id" ref="model_training_course"/>  
  <field name="domain_force">[( 'teacher_id.user_id', '=')<br>  
    <field name="groups" eval="[(4, ref('group_training_te<br>  
</record>
```

Manager bypass group

```
<field name="groups"  
  eval="[(4, ref('group_training_manager'))]" />
```

4.6 Groups on Actions

- Groups on an action limit who can launch it.
- They help hide reports or tools from unrelated roles.
- They do not replace model permissions.

Action with group

```
<record id="action_training_report" model="ir.actions.act_  
  <field name="name">Training Report</field>  
  <field name="groups_id" eval="[(4, ref('group_training_  
</record>
```

Server action group

```
<field name="groups_id"  
  eval="[(4, ref('base.group_system'))]" />
```

4.7 Groups on Fields

- A field can be visible only to selected groups.
- Use this for sensitive or advanced fields.
- Server permissions still protect the underlying model.

View field group

```
<field name="internal_note"  
      groups="training.group_training_manager"/>
```

Python field group

```
internal_note = fields.Text(  
    groups="training.group_training_manager"  
)
```

4.8 XML Data noupdate

- noupdate protects data from being overwritten on module update.
- Use it for user-edited configuration records.
- Do not use it for views you expect to upgrade.

Protected data

```
<data noupdate="1">  
  <record id="seq_training_course" model="ir.sequence">  
    <field name="name">Course Sequence</field>  
  </record>  
</data>
```

Normal data

```
<data>  
  <record id="view_training_course_form" model="ir.ui.view">  
</data>
```

4.9 CSV vs XML Data

- CSV is compact for many similar rows.
- XML is better for nested records and references.
- Choose the format that future maintainers can review.

CSV access row

```
access_course_user,course user,model_training_course,base.
```

XML menu record

```
<menuitem id="training_menu_root"  
          name="Training"  
          sequence="20"/>
```

4.10 Sequence Basics

- ir.sequence generates readable document numbers.
- Use a code that your model can call.
- Keep prefixes short and meaningful.

Sequence XML

```
<record id="seq_training_course" model="ir.sequence">  
  <field name="name">Course</field>  
  <field name="code">training.course</field>  
  <field name="prefix">CRS/</field>  
</record>
```

Python use

```
vals["name"] = self.env["ir.sequence"].next_by_code(  
    "training.course"  
)
```

4.11 External IDs

- External IDs let XML and CSV records refer to each other.
- They must stay stable across upgrades.
- Use clear names with the module purpose included.

Reference a group

```
<field name="group_id" ref="training.group_training_teachers"/>
```

Reference a model

```
<field name="model_id" ref="model_training_course"/>
```

5 Section 5

Reports, Templates, and Automation

5.1 QWeb Report Basics

- QWeb reports render HTML that can become PDF.
- A report action connects the model to the template.
- Keep report templates simple and printable.

Report action

```
<record id="action_report_course" model="ir.actions.report">  
  <field name="name">Course Report</field>  
  <field name="model">training.course</field>  
  <field name="report_type">qweb-pdf</field>  
  <field name="report_name">training.report_course</field>  
</record>
```

Template shell

```
<template id="report_course">  
  <t t-call="web.html_container">  
    <t t-foreach="docs" t-as="doc"/>  
  </t>  
</template>
```

5.2 t-field and t-out

- t-field renders an Odoo field with formatting.
- t-out prints a computed expression safely.
- Prefer t-field for dates, money, and relational fields.

Formatted field

```
<span t-field="doc.start_date"/>
```

Expression output

```
<span t-out="doc.name.upper()"/>
```

5.3 QWeb If

- t-if conditionally shows a piece of template.
- Use it for optional values and simple branches.
- Keep business decisions in Python when they become complex.

Optional note

```
<p t-if="doc.note">  
    <span t-field="doc.note"/>  
</p>
```

State message

```
<strong t-if="doc.state == 'done'">Completed</strong>
```

5.4 QWeb Foreach

- t-foreach repeats markup for each record or value.
- Use t-as to name the current item.
- Keep loop rows small for readable reports.

Loop lines

```
<tr t-foreach="doc.line_ids" t-as="line">  
  <td><span t-field="line.name"/></td>  
</tr>
```

Loop numbers

```
<li t-foreach="[1, 2, 3]" t-as="number">  
  <span t-out="number"/>  
</li>
```

5.5 Paper Format

- Paper formats define page size and margins.
- Attach one when a report needs special layout.
- Test printed output, not only the browser preview.

A4 format

```
<record id="paperformat_training" model="report.paperformat">  
  <field name="name">Training A4</field>  
  <field name="format">A4</field>  
  <field name="margin_top">20</field>  
</record>
```

Attach to report

```
<field name="paperformat_id"  
  ref="paperformat_training"/>
```

5.6 Email Template

- Email templates generate messages from records.
- Use placeholders for record-specific values.
- Keep the subject clear and searchable.

Template record

```
<record id="email_course_confirmed" model="mail.template">  
  <field name="name">Course Confirmed</field>  
  <field name="model_id" ref="model_training_course"/>  
  <field name="subject">Course {{ object.name }} confirm</field>  
</record>
```

Body expression

```
<field name="body_html" type="html">  
  <p>Hello <t t-out="object.teacher_id.name"/>.</p>  
</field>
```

5.7 Server Action

- A server action runs server-side logic from the UI or automation.
- Use it for small administrative actions.
- Move complex logic into model methods.

Call Python method

```
<record id="server_action_confirm_courses" model="ir.action" data-bbox="508 233 1000 429">  
  <field name="name">Confirm Courses</field>  
  <field name="model_id" ref="model_training_course"/>  
  <field name="state">code</field>  
  <field name="code">records.action_confirm()</field>  
</record>
```

Bind to model

```
<field name="binding_model_id" ref="model_training_course" data-bbox="508 537 1000 602"/>  
<field name="binding_view_types">list,form</field>
```

5.8 Header Buttons and Security

- Header buttons should match the record workflow.
- Use groups to hide actions from users who cannot run them.
- The Python method must still check important rules.

Manager-only button

```
<button name="action_approve"  
        type="object"  
        string="Approve"  
        groups="training.group_training_manager"/>
```

State-based button

```
<button name="action_done"  
        type="object"  
        string="Done"  
        invisible="state != 'confirmed'"/>
```

5.9 Manifest Data Order

- Data files load in the order listed in the manifest.
- Load security before views that use security groups.
- Load reports after their templates and dependencies exist.

Manifest order

```
"data": [  
  "security/groups.xml",  
  "security/ir.model.access.csv",  
  "views/course_views.xml",  
]
```

Report files

```
"data": [  
  "report/course_templates.xml",  
  "report/course_reports.xml",  
]
```

NEXT STEP

Try the examples on your machine

Then open the next module deck

Merit Advisory

05 · Views & Security

